Operacijski sistemi

Procesi

Jurij Mihelič, FRI, Uni LJ

Vsebina

- Delo s procesi v lupini
- Sistemski klici za delo s procesi
- Unix-podobni sistemi

Delo s procesi v lupini

- Pregled procesov

`ps`	… izpis trenutnih procesov
`pidof`	… iskanje pid procesa po imenu
`pgrep`	… napredno iskanje pida procesa po imenu
`pstree`	… izpis hierarhije procesov
`top`	… interaktiven pregled procesov

Delo s procesi v lupini

- Delo s procesi

`echo $$`	… PID lupine
`echo $PPID`	… PPID lupine
`echo $BASHPID`	… PID procesa (lahko gre za podlupino)
`sleep` *seconds*	… spanje za določen čas
`wait`	… čakanje vseh otrok, da se končajo
`wait` *PID*	… čakanje, da se otrok PID konča
`exit` *status*	… končanje izvajajočega se procesa
`kill` *PID*	… ukinjanje procesa PID

```
echo $PPID $$ $BASHPID
(echo $PPID $$ $BASHPID )
( (echo $PPID $$ $BASHPID ) )
```

Delo s procesi v lupini

- Sistem `/proc/`*PID*
  - informacije o procesu št. PID
  - `man proc`

Zanimive datoteke

- `exe` … simbolična povezava na izvršljivo datoteko procesa
- `cmdline` … ukazna vrstica zagona procesa
- `environ` … okoljske spremenljivke procesa
- `cwd` … simbolična povezava na delovni imenik procesa
- `root` … simbolična povezava na korenski imenik procesa
- `limits` … omejitve virov procesa (`ulimit`)
- `stat` … statusne informacije procesa, programski dostop
- `status` … statusne informacije procesa, človeku prijazno
- `fd/` … imenik, simbolične povezave na odprte datoteke
- `fdinfo/` … imenik, informacije o deskriptorjih datotek
- `task/` … niti procesa

Delo s procesi v lupini

- Demo: processinfo.sh
  - Napiši skripto, ki izpiše informacije o procesu s podanim pidom

Primer izpisa

```
Izvršljiva datoteka: /usr/bin/bash
Ukazna vrstica: -bash
Delovni imenik: /home/jure/os
Korenski imenik: /home/jure/os
Ime procesa: bash
PID: 1700
PPID: 1699
Št. niti: 1
Odprte datoteke
  0 -> /dev/pts/0
  1 -> /dev/pts/0
  2 -> /dev/pts/0
  255 -> /dev/pts/0
```

Sistemski klici za delo s procesi

- Ponovitev

fork()	… ustvaritev novega procesa
exit()	… končanje tekočega procesa
wait()	… čakanje na končanje poljubnega otroka
waitpid(…)	… čakanje na končanje določenega otroka
execXX(…)	… izvedba programa

Sistemski klici za delo s procesi

- Sistemski klic: int fork()

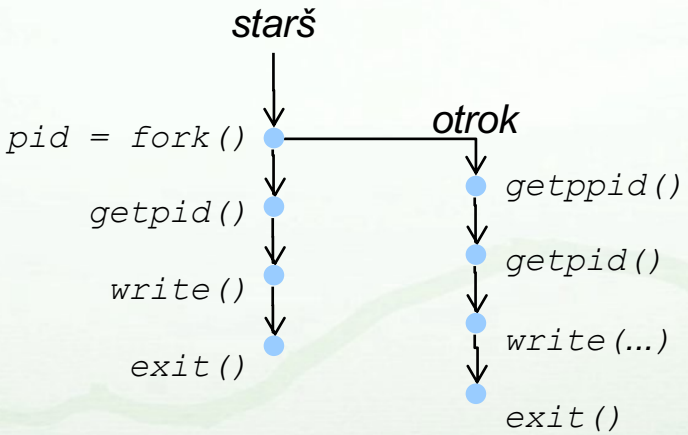
```
int pid = fork();
if (pid > 0) {
        // STARŠ
} else if (pid == 0) {
        // OTROK
} else { // pid < 0
        // NEUSPEH
}
```

```
#include <stdio.h>

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid == 0)
        printf("Sem otrok %i s staršem %i.\n", getpid(), getppid());
    else
        printf("Sem starš %i z otrokom %i,\n", getpid(), pid);
}
```

Sistemski klici za delo s procesi

- Sistemski klic: int fork()

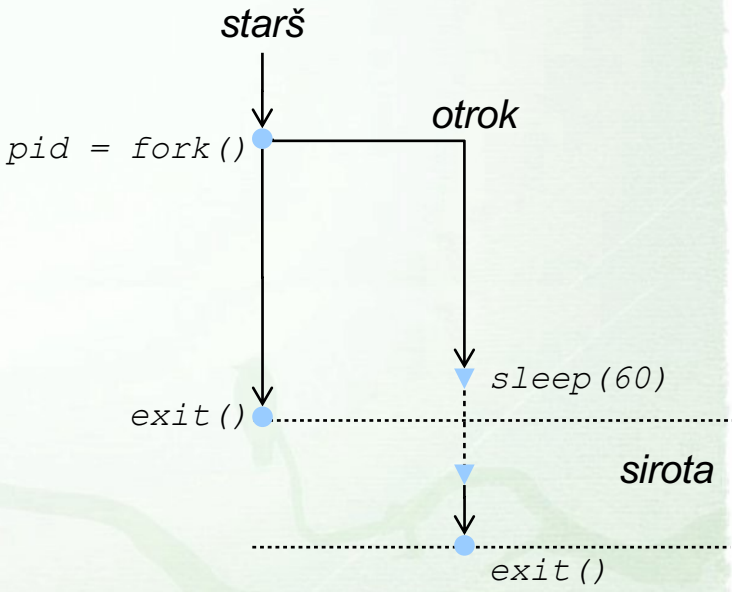
```
#include <stdio.h>

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid == 0)
        printf("Sem otrok %i s staršem %i.\n", getpid(), getppid());
    else
        printf("Sem starš %i z otrokom %i,\n", getpid(), pid);
}
```



Sistemski klici za delo s procesi

- Nastanek sirote

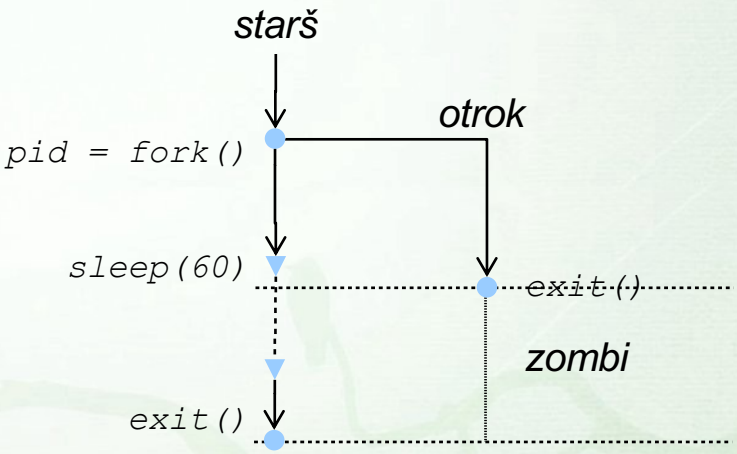
```
#include <stdio.h>

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid == 0)
         // otrok zaspi za 60 sekund
        sleep(60);
}
```



Sistemski klici za delo s procesi

- Nastanek zombija

```
#include <stdio.h>

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0)
        perror(argv[0]);
    else if (pid > 0)
         // starš zaspi za 60 sekund
        sleep(60);
}
```



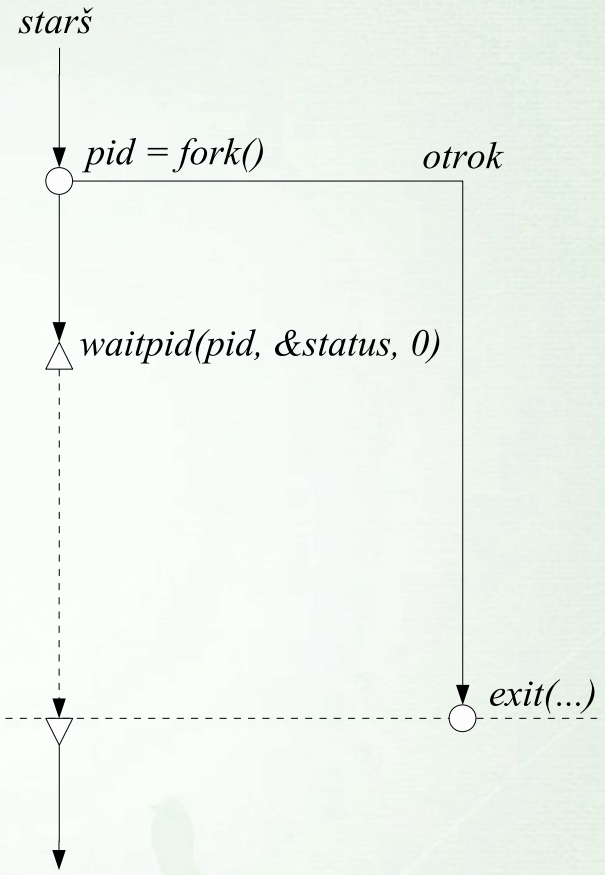
Unix-podobni sistemi

- **Prevzem** izhodnega status otroka – wait
  - in **čakanje** na dokončanje, če še ni končan
    - int waitpid(pid, &status, opcije)
    - int wait(&status)
  - man waitpid
  - makro WIFEXITED(status)
    - se je program končal z exit()?
  - makro WEXITSTATUS(status)
    - iz spremenljivke status se izlušči izhodni status

```
if (WIFEXITED(status))
    status = WEXITSTATUS(status));
```

Unix-podobni sistemi

- Časovni diagram procesov
  - fork
    - stvaritev otroka
    - otrok dobi vse podatke od starša
  - exit
    - oddaja statusa staršu
  - wait
    - prejem statusa



Unix-podobni sistemi

- Demo: starsndots.c
  - Napiši program v C, kjer
    - starš
      - ustvari 3 otroke
      - izpiše 10 zvezdic, vsako sekundo eno zvezdico
      - nato počaka na otroke in prevzame njihove izhodne statuse
      - izpiše prevzete statuse
    - 3 otroci
      - izpisuje 20 pikic/vejic/podpičij, vsako sekundo 1
      - nato se konča s statusem 42 + št. otroka

Unix-podobni sistemi

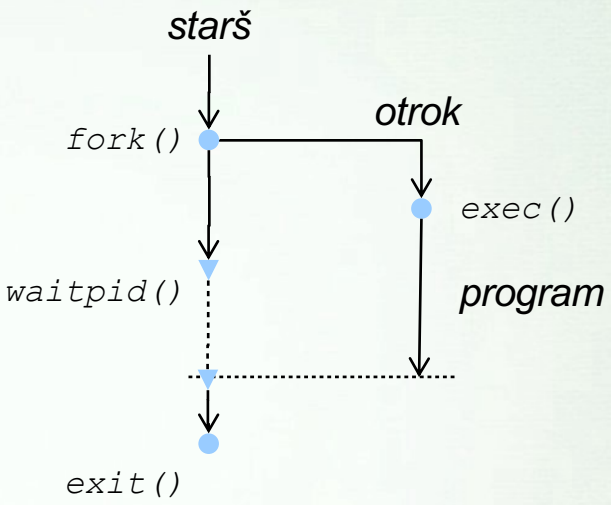
- Sistemski klic exec[lv][pPe]?(...)
  - podamo pot do izvršljive datoteke oz. ukaza
  - pripone
    - l – argumenti ukaza so podani kot argumenti funkcije
    - v – argumenti ukaza so podani posebej v tabeli
    - p – uporaba $PATH pri iskanju izvršljive datoteke
    - P – podamo tudi iskalno pot
    - e – okoljske spremenljivke so podane v tabeli

funkcija
execl(exe, arg0, ...)
execlp(exe, arg0, ...)
execle(exe, arg0, ..., 0, envp)
execv(exe, args)
execvp(exe, args)
execvP(exe, path, args)
execve(exe, args, envp)

Procesi v C

- Zagon programa

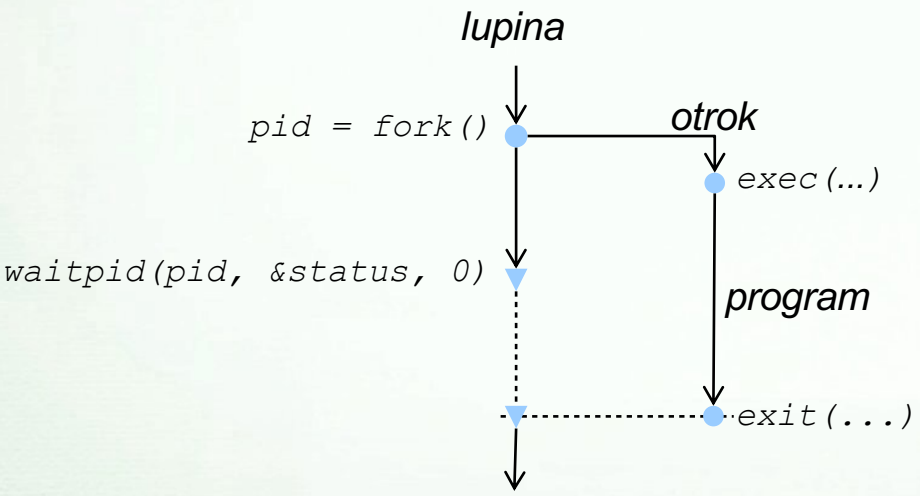
```
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main(int argc, char* argv[]) {
    int pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror("exec");
        exit(EXIT_FAILURE);
    } else {
        int status;
        if (waitpid(pid, &status, 0) < 0) {
           perror("waitpid");
           exit(EXIT_FAILURE);
        }
        if (WIFEXITED(status))
           printf("Izhodni status otroka: %i\n",
               WEXITSTATUS(status));
    }
    exit(EXIT_SUCCESS);
}
```



Procesi v lupini

- Zagon programa v ospredju

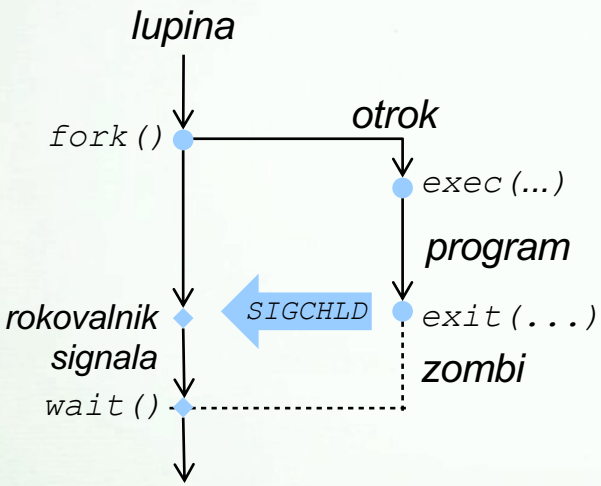
```
ls
```



Procesi v lupini

- Zagon programa v ozadju

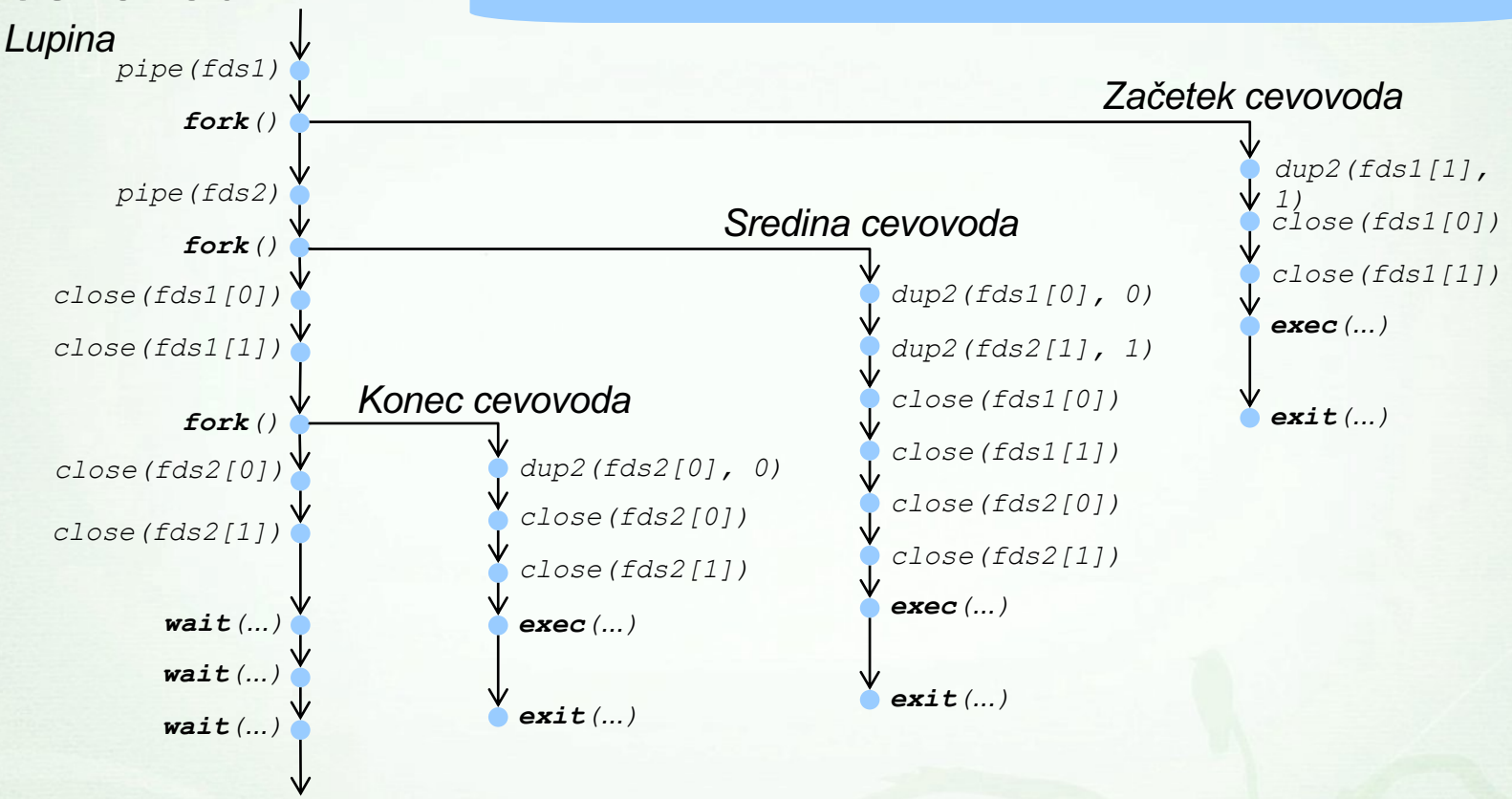
```
xeyes &
```



Procesi v lupini

- Cevovod

```
cat /etc/passwd | cut -d: -f7 | sort -u
```



Procesi v lupini

- Zagon zunanjega ukaza
  - `fork(), exec(), waitpid()`

```
ls
```

- Zagon zunanjega ukaza v ozadju
  - `fork(), exec()`

```
xeyes &
```

- Izvajanje v podlupini: `(` *vgrajeni_ukaz* `)`
  - `fork(), waitpid()`

```
( read line; echo $line )
```

- Izvajanje v podlupini v ozadju: `(` *vgrajeni_ukaz* `) &`
  - `fork()`

```
( echo {24..42} >fri.txt ) &
```